Semantic Analysis

CS/COE 1622

Jarrett Billingsley

Class Announcements

- Ö

Semantics

What does it all MEAN

- **semantics** means… meaning.
- a language's semantics are the rules that make it *that language*.

```
int x = 10;
printf("x = " + x);
```

this code lexes and parses **exactly the same** in both C and Java.

but the *semantics* differ wildly.

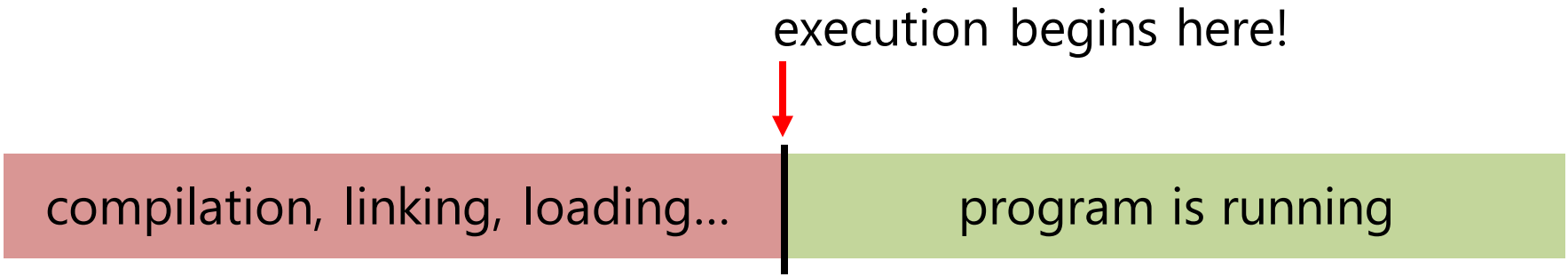
in Java, this says: "create a new string object that is the concatenation of `"x = "` and the result of calling **`x.toString()`**."

in C, this says: "calculate a pointer starting at the address of `"x = "` and adding the value in **x** times the size of one **`char`**."

the semantics define **what each piece of the language *does*.**

Static vs. Dynamic

- you'll see these words used a lot in talking about semantics
- if we look at a timeline of the process of compiling and running...



static means something that is done **before execution begins,** and it's only done *once*.

you'll also hear it called **"compile-time."**
(or "link-time" or "load-time")

dynamic means something that is done **during execution,** possibly *over and over*.

you'll also hear it called **"runtime."**

Abstraction

- HLLs provide programmers with useful sets of **abstractions.**
- semantics are about **enforcing the rules** of those abstractions…
  - even if the target CPU doesn't make a distinction!

```
int i       = 0;
Object o    = null;
char c      = '\0';
boolean b = false;
```

in this Java, all four lines here will probably compile down to the exact same code, something like:

```
sw  $zero, i
```

the CPU doesn't know or care about the difference between these types, but **the language does.**

Compile-time and runtime guarantees

- the compiler is responsible for enforcing many of the language's rules, but those rules can extend **beyond the compiler's reach!**

```
int[] a = new int[5];
a[10] = 0; // oops
```

a Java compiler *might* be able to catch *this* contrived case.

but in the general case, this error **can't be detected until runtime.**

static checks done by the compiler try to catch as many mistakes as possible...

...but they can't catch every mistake. doing so would require **solving the halting problem.**

still, doing what static checks we can is better than leaving *all* the semantic analysis until runtime!

Semantic analysis can get tricky

- in most nontrivial languages, semantic analysis is not just "one step."
  - there may be **several phases** or **passes** of semantic analysis…
  - and sometimes, those passes can be run **multiple times,** or be **mutually recursive** with one another!
- the toy language we're working with won't get *too* scary to analyze…
  - but it's good to keep in mind that even seemingly-simple features can cause the complexity of semantic analysis to explode.
- one of the most powerful tools we have to define the rules and boundaries of a language is…

Types

Types and typing

- HLLs give us several kinds of **values** to work with.
- we classify these values according to their **type.**

a **type** is a set of valid **values...**

```
{ 0, 1, -1, 2, -2, ... }
```

...along with a set of **operations** that can be performed on them.

```
{ +, -, *, /, %, parse, ...}
```

types set the **boundaries** on what you can and can't do within the language.

You have to listen to the notes they don't play

- down at the CPU level, types don't really exist.
  - you can do almost anything with anything! the CPU doesn't care!
- we say that assembly is **untyped.**

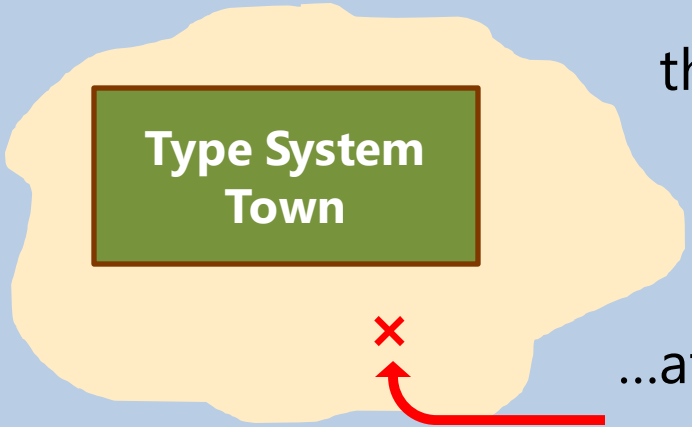
```
li   t0, 10        # oh, t0 is an int
sw   zero, (t0)    # wait we're using it as an address?
li   t0, 'a'       # uhh now it's a character?
jr   t0            # and now we're jumping to it??
```

- in an untyped language, **the programmer can do anything.**
  - but is that really a good idea...?

A vast sea of garbage

- a program is **any sequence of instructions.**
- so if we consider *all possible sequences of instructions…*

almost all of them do *nothing useful at all.*



this is **Useful Program Island.**

type systems keep you safely on the island…

…at the cost of preventing you from being able to write *all* useful programs. like this one.

but maybe these kinds of programs are too hard for humans to understand anyway, so no big loss?

The soul of a programming language

- the type system affects **every operation in the language.**
- the type system decides…
  - what kinds of variables can exist
  - what operators and methods you can use on them
  - how those operators do their work
  - how control structures work
- the main difference between C and Java is their **type systems.**
  - even though their syntax can look superficially similar…
  - the boundaries drawn by their type systems are completely different.

Type systems

Primitive types

- every language considers some types to be **primitive,** or **fundamental.** they cannot be broken into any simpler parts.

some, like **ints** and **floats**, are derived from the CPU's abilities.

```
lw  $t0, int_var
l.s $f0, float_var
```

others, like **bools** and **chars**, are simple *abstractions* on top of those...

but they help us encode *intent* into our programs, and intent helps us better understand our own code (and the code that others write).

```
int isDone = 0; // ??
bool isDone = false; // better.
```

Compound types

- **compound** types are composed of two or more primitive types.
  - you can think of primitive and compound types like **atoms and molecules:** multiple atoms make up a molecule.

```
struct Point {
  x: i32,
  y: i32,
}
```

structs and **classes** are common ways to let the programmer define their own compound types.

```
let p: (i32, i32) = (4, 5);
```

tuples are a little obscure, but work similarly to structs.

```
let a: [i32; 3] = [1, 2, 3];
```

arrays also count, but they're a little more interesting…

Generic types

- a **generic** type is a special kind of function which **takes types as its inputs,** and **produces a type as its output.**
  - we call that a **type constructor.**
- if that sounds really out there, well, how about `ArrayList`?
  - it takes *type arguments* in angle brackets: `ArrayList<Integer>`
- and if you give it different type arguments, you get a different type.
  - `ArrayList<Integer>` is different from `ArrayList<Double>`.
- another example in Java is the square brackets: `[]`
  - you can't just have a `[]`. it has to be an array *of something*.
- you can put **any** type before `[]` and get a new type, so it's a type constructor.
  - `int[]` is a new type. `int[][]` is another new type. and so on, forever!

And further still...

- there are even *type constructors that can take **values** as arguments.*
- there's a very concrete example for that: **arrays**
  - see, an array *has a length*
  - but... what if that length were **part of the type?** like `int[10]`?
- could you make a language where you could *never have an **`ArrayIndexOutOfBoundsException`**?*
  - **yes!**
  - it's possible to track the length of every array and the **range of every integer** at compile time, and ensure they can't happen.
- but there's a big downside to this:
  - in the *general* case, **these type systems are undecidable (meaning you have to solve the halting problem).**
    - but in some limited cases, it *is* decidable, and maybe useful?

Naming

Hello, my name is...

- **computers** don't know anything about names, language, or strings...
  - but **humans** love words. we can't get enough of em.
- one of the reasons people very quickly invented assembly language was to be able to **name things in their programs,** because dealing with addresses is confusing, tedious, and error-prone.
  - can you imagine having to refer to everything by a number?
  - and where changing one part of the program changed the numbers of everything after it, and so you had to go back and change all the references to all those numbers...
    - it was not doable.

Scoping and name resolution

- **name resolution** matches names to the things they refer to.
- **scoping** determines which names are candidates during resolution.

```
void main() {
  int m = 5;
  f();
}

void f() {
  printf("m = %d\n", m);
}
```

you know that this is wrong, but not every language works like this!

in some languages, **you can access the local variables of any function on the call stack.**

this used to be somewhat common in older languages, but eventually we realized "wow that's confusing as hell"

we've *kind of* settled on a common set of name resolution rules, but some languages can still surprise you.

Static vs. dynamic name resolution

- some names' **referents** can't be decided until runtime.

```
class A { String toString() { return "A"; } }
class B { String toString() { return "B"; } }

Object[] objs = new Object []{ new A(), new B() };
for(Object o : objs)
    System.out.println(o.toString());
```

which **`toString()`** does **`o.toString()`** refer to?

on the other hand, all the class names, variable names etc. are **statically** resolved.

in some languages, *all* names are dynamically resolved.

Round and round

- name resolution can create a complex or even **cyclical** graph of dependencies between the parts of a program.

```
class A {
  void print(B b) { println(b.value()); }
  int value()     { return 10; }
}
class B {
  void print(A a) { println(a.value()); }
  int value()     { return 20; }
}
```

A depends on B, and B depends on A.

I can't check that all of A's code is correct until I check B's...
but I can't check that B's is correct until I check A's...???

well. it's possible to do things in **phases,** like I said before.

Scoping

What is it?

- the programmer can declare **named things** like variables, etc.
- the **scope** is *where* those names can be seen in the program.

```
if(x == 10) {
  int y = 20;
  printf("%d", y);
}
```

this is alright!

```
printf("%d", y);
```

this is **not**.

well, that's how you *learned* it. **not every language has the same rules.**

```
# Python
if x == 10:
  y = 20
  print(y) # works!

print(y)   # works!
```

Static vs. dynamic

- the *majority* of languages use **static scope:** the **syntactic structure** of the language defines where a name can be seen.
- Python and many other dynamically-*typed* languages are instead **dynamically *scoped:*** the variable is not looked up until runtime.

```
# Python
def f():
  print(g)

f()     # runtime error!
g = 10 # g is a global
f()     # prints 10
```

dynamic scope can lead to really confusing problems; the same piece of code can either work or not work **based on what variables are declared around it.**

but I think most dynamic languages do it like this cause it's easy to implement!

Shadowing

- names aren't always unique.
- sometimes this is clearly a mistake, but in other cases...

```
class A {
  int x;
  A(int x) {
    // which x?
    x = 5;
  }
}
```

this refers to...
that argument.

in this case we say that the local variable **shadows** the member variable.



Shadowing isn't always a mistake

- sometimes it just makes things more convenient.
  - if you write a piece of code, and then change the environment around it (like adding a global), you don't want it to suddenly change behavior or get new compiler errors!
- or in Rust, where variables are immutable by default, this is a common pattern:

```
let ast = parse(lex(input)?)?;
let types = typecheck(ast)?;
let ast = const_fold(ast, types)?;
let ast = inline_calls(ast, types)?;
codegen(ast, types)?;
```

- subsequent declarations of **`ast`** **shadow** the previous ones.

Evaluation and Execution

Evaluation and Side Effects

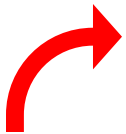
- **evaluation** means figuring out the value that an expression has.
  - it might be trivial, like for literals: `10` evaluates to… `10`!
- but it might require execution of an unknown amount of code…
- and in the process, that code may also produce **side effects:** changes to things outside of the code's local variables, like:
  - modifying global variables
  - modifying objects that were passed to it by reference
  - performing input or output
  - making your computer explode
- side effects make it hard for us to reason about code…
  - they can cause the same piece of code to do different things depending on how many times you run it.
- but without side effects, code can't do anything useful!
  - imagine having no input or output!

Evaluation Order

- here's some Java code:

```
int f(int x) { println("got " + x); return x; }
int add(int a, int b) { return a + b; }
println("3 + 5 = " + add(f(3), f(5)));
```

- **which one does it print?**



```
got 3
got 5
3 + 5 = 8
```

```
got 5
got 3
3 + 5 = 8
```

we have an *expectation* that code executes left-to-right, so we expect this left one… and this is correct. *for Java*.

evaluation order is defined by a language's semantics, and Java happens to define this order, but **not all languages agree!**

Eager vs. Lazy

- a less common distinction is **eager versus lazy evaluation.**

```
int x = someLongComputation();
System.out.println("done.");
System.out.println("x = " + x);
```

- in the above Java code, **someLongComputation()** is called, and its return value is assigned into x **after it returns.**
  - this is **eager evaluation,** in which the function call occurs *at the point where you write it.*
- but another way is **lazy evaluation,** in which the variable declaration doesn't call the function, *the use of x on the last line does.*
  - that might sound bizarre, but is actually very popular in **functional** languages, where **side-effect-free functions** are the norm!
- eager evaluation is by far the more popular strategy today, but…

You never knew you were using it

- in this code:

```
if(condOne() && condTwo()) { println("woo"); }
```

- if **`condOne()`** returns false, does **`condTwo()`** ever run?
  - nope.
- *this is lazy evaluation*.
  - in all C-style languages, **`&&`** and **`||`** **lazily** evaluate their second operand: they only execute that code if needed.
- lazy evaluation can be really useful when using higher-order functions (passing functions to other functions), too.
- so don't write it off as academic nonsense!

Languages without Semantic Analysis?

Who needs semantic analysis anyway

- depending on how the language is designed, **we may not have to do any semantic analysis *in the compiler*.**

```
def test():
    x = 10
    x = "hello"
    x = print
    x("hi!")

test()
```

Python is a **dynamically-typed language.**

that means the semantic rules of its type system are checked **at runtime,** as the code executes.

this is totally valid Python code, but there's **no way for the compiler to tell.** so, it… doesn't.

compilers for dynamic languages can be pretty simple: lex, parse, and then… well, right to execution!

Executing the AST

- we saw this already with the **ast_math** example!
  - once you build the AST, you just write some recursive methods to **visit all the AST nodes in order,** evaluating them as you go.
  - it's not hard to extend this concept to AST nodes for loops, conditionals, function calls etc.
- this is an **AST interpreter,** and is a really quick-and-dirty way to get *something* running.
  - from what I understand, Ruby used exactly this execution strategy for over 10 years!
- but as you might imagine, *it is not fast at all.*
  - consider all the interpreter code that needs to run just to, say, add a couple numbers together.
- but this topic is better left for after the midterm…